

Technical Security Analysis Portfolio

Offline Environment Analysis and Runtime Logic Reengineering of an ARM64 Mobile Game Engine

Responsible Disclosure Notice

This document presents a controlled security research case study based on offline binary analysis of an ARM64 Android application. Target identifiers, developer/platform names, function names, and virtual offsets have been intentionally generalized or obfuscated to support responsible disclosure and prevent direct identification of the production target. The techniques described are intended solely for authorized security research, defensive engineering, and educational analysis. No production credentials, user accounts, or third-party services are targeted or disclosed.

1. Abstract & Executive Summary

1.1 Abstract

This whitepaper documents a controlled reverse-engineering and security analysis of a stripped ARM64/AArch64 Android native game engine. The research focused on understanding how the application handled offline-session restrictions, local configuration parameters, and internal premium-currency transaction logic.

The target binary was heavily stripped, providing no meaningful debugging symbols and exposing internal routines primarily through generic identifiers. Initial analysis concentrated on application behavior occurring when the game was executed without network connectivity.

The investigation identified two significant areas of interest:

- An offline-session state-management mechanism that enforced a predefined runtime duration before directing the application toward a premium authentication/storefront flow.
- An internal transaction-processing routine whose arithmetic and validation behavior could be manipulated through instruction-level modification.

Static analysis demonstrated that simply suppressing the visible authentication dialog did not restore normal execution. Instead, the application entered a persistent black-screen or soft-lock condition, indicating that the user interface was only one component of a larger validation state machine.

Further examination of the native library's .rodata section uncovered an embedded JSON configuration containing an offline-session duration parameter. Tracing this configuration through the parsing pipeline revealed the relationship between the static configuration, runtime session state, and the enforcement mechanism.

A separate analysis of the internal transaction engine demonstrated how ARM64 register-level arithmetic affected balance validation. Controlled instruction modification showed that replacing selected instructions could alter the interpretation of transaction values and session-duration parameters.

The case study therefore illustrates an important security principle:

Client-side enforcement mechanisms should not be treated as trustworthy security boundaries when critical authorization, entitlement, currency, or subscription decisions can be reconstructed and modified entirely within the client binary.

1.2 Executive Summary

The research identified several architectural weaknesses associated with relying on locally enforced security logic:

- Critical offline-session parameters were recoverable from native binary resources.
- Authentication-related UI behavior was coupled to deeper runtime state-management logic.
- Transaction validation depended substantially on client-side arithmetic.
- Runtime values were stored in conventional memory representations rather than protected or cryptographically authenticated representations.

- Native code integrity was not sufficiently protected against localized instruction modification.
- Static configuration and executable logic could be correlated through cross-reference and data-flow analysis.

2. Static Analysis & Architectural Tracing

2.1 Target Architecture

The analyzed component was an Android native library compiled for:

Property	Observed Characteristic
Platform	Android
Architecture	ARM64 / AArch64
Binary Type	Native shared library
Symbol State	Heavily stripped
Function Naming	Generic sub_XXXXXX identifiers
Primary Analysis	Static + runtime behavioral analysis

The absence of meaningful symbols significantly increased the analytical workload. Consequently, the research relied on a combination of:

- String discovery
- Cross-reference analysis
- Control-flow graphs
- Function-call tracing
- Register tracking
- Data-flow analysis
- Runtime observation

- Instruction-level inspection

2.2 UI Obstacle and Soft-Lock Tracing

The first analytical objective was to understand the application's behavior when authentication validation could not be completed.

Localized error-related strings were identified within the native binary. Cross-referencing these strings led to a dialog-generation routine provisionally designated: `sub_AuthDialogSpawner`

with the research virtual offset: `0x003FF000`

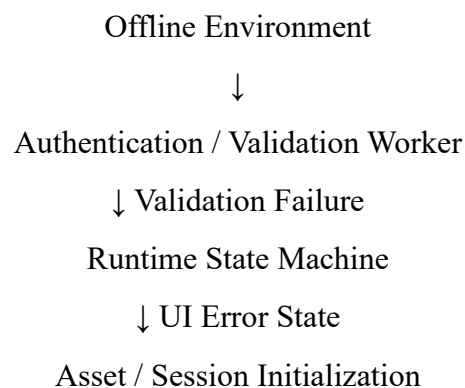
The initial hypothesis was that this function represented the primary enforcement point responsible for presenting the authentication-related error state.

A controlled early-return modification was therefore evaluated conceptually using an ARM64 pattern equivalent to:

- `MOV X0, #0`
- `RET`

The visible authentication dialog was successfully suppressed during testing. However, the application subsequently entered a persistent black-screen state.

This result was particularly important because it disproved the initial assumption that the dialog itself constituted the complete enforcement mechanism. Instead, the observed behavior suggested a deeper dependency chain





Blocking Condition

Suppressing the UI therefore removed the visible symptom without satisfying the underlying state transition.

This represents a common reverse-engineering pattern, a user-visible security warning may only be the terminal representation of a deeper internal state rather than the actual security control.

2.3 Embedded Configuration Discovery

Following the unsuccessful UI-level approach, analysis moved toward static configuration data.

The .rodata section of the native library was inspected for parameters associated with offline execution. A plaintext JSON configuration template was identified around the generalized research offset: 0x009D8000

The configuration contained a parameter equivalent to: "offline_session_duration": 3600

The value 3600 corresponds to: 3,600 seconds

This discovery provided a strong correlation between the observed one-hour offline restriction and a statically embedded runtime parameter.

2.4 Configuration Data-Flow

Cross-reference analysis associated the configuration with a parsing routine designated: sub_ConfigParser

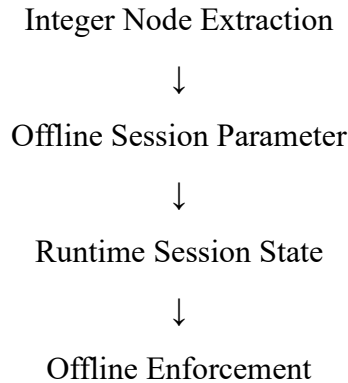
The relevant execution model could be represented as

Embedded JSON



Configuration Parser





This tracing was significant because it established a direct relationship between:

- Static binary data,
- Configuration parsing,
- Runtime state initialization, and
- Offline-session enforcement.

From a defensive standpoint, this demonstrates why security-sensitive configuration should not automatically be assumed to be trustworthy simply because it is located inside a compiled native binary.

3. Vulnerability Exploitation & Assembly Reengineering

3.1 ARM64 Register-Level Analysis

The second phase examined the internal transaction-processing mechanism.

The relevant routine was provisionally identified as: `sub_ProcessTransaction`

with a generalized research offset of: `0x00397000`

The analysis identified an internal accumulator associated with transaction values: `X20`

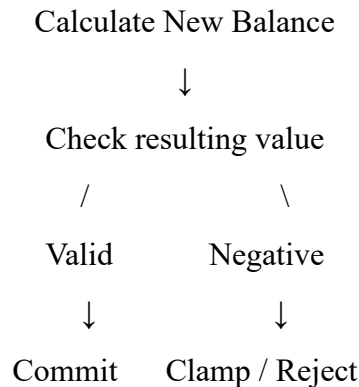
The transaction model could be simplified as: $\text{New Balance} = \text{Current Balance} + \text{Transaction Value}$

where a purchase operation generated a negative transaction value: $\text{Transaction Value} = -\text{Asset Cost}$

Under normal execution: $\text{Current Balance} + (-\text{Asset Cost})$ would reduce the balance.

3.2 Negative-Balance Validation

The transaction-processing path contained conditional selection/inversion behavior associated with balance validation, including ARM64 instructions from the CSEL / CSINV family. Conceptually, the validation operated as:



The offline starting state was particularly relevant because the initial balance could be zero.

Consequently: $0 + (-\text{Cost}) \rightarrow \text{Negative Result}$ could trigger the application's defensive balance handling.

3.3 Instruction-Level Manipulation

A controlled modification was evaluated at the relevant execution boundary, represented in the research documentation by the generalized offset: 0x00930000

The original instruction sequence was replaced conceptually with: NEG X20, X20

The ARM64 NEG instruction performs two's-complement negation: $X20 = -X20$

Therefore, a negative incoming transaction value could be transformed into a positive value before subsequent arithmetic processing.

The resulting computational model became approximately:

- Original: Transaction = -Cost
- Modified: Transaction = $-(-\text{Cost}) = +\text{Cost}$

The modified value could then participate in the subsequent addition operation represented by the research notation: ADDS X8, X9, X20

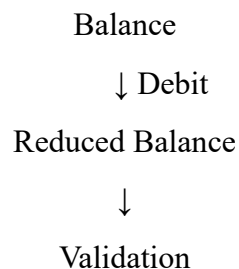
The key security observation is not the specific instruction itself, but the architectural weakness it exposed, the client trusted locally computed transaction values sufficiently that modifying a single arithmetic operation could alter the resulting financial state.

3.4 Impact on the Wallet State

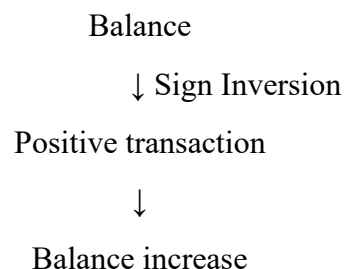
The modified execution path demonstrated that a value originally interpreted as a debit could instead be interpreted as a credit.

The resulting state transition could be represented as:

- Normal Path:



- Modified Path:



This illustrates a fundamental weakness in client-authoritative virtual-currency systems. If the authoritative financial state exists entirely within the client, then an attacker capable of modifying native instructions may be able to alter the state without needing to defeat a conventional authentication mechanism.

3.5 Session Configuration Reengineering

The session-duration path was subsequently examined.

The configuration parser was associated with a generalized virtual offset: 0x00DC0000

and the integer-extraction routine was designated: sub_ExtractNodeInteger

The parser extracted the embedded duration value and subsequently committed it to an internal structure through a store operation equivalent to: STR W0, [X20,#0x1E8]

The relevant field therefore represented a runtime session-limit value.

3.6 Register-Level Configuration Substitution

The research identified a call site represented by: BL sub_ExtractNodeInteger

The instruction was experimentally replaced with a direct immediate assignment: MOV W0, #0xFFFF

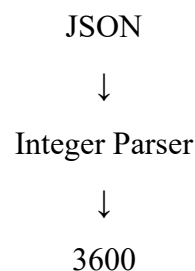
The functional effect was to bypass the normal integer-extraction result and provide a predetermined value directly through the ARM64 return-value register.

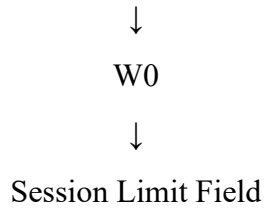
The resulting value was: 0xFFFF = 65,535 or approximately: 18.2 hours

This created a modified runtime configuration in which the session-duration field received the substituted value rather than the original configuration-derived value.

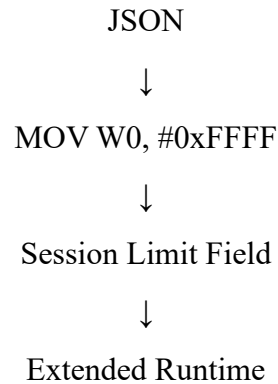
The resulting data flow can be represented as:

- Original:





- Modified:



3.7 Security Significance

The experiment demonstrated that a relatively small instruction-level modification could alter a security-sensitive runtime parameter. This reveals three architectural weaknesses:

- Configuration Trust, the application treated a locally stored configuration value as authoritative.
- Code Integrity, the application did not sufficiently prevent localized native-code modification.
- Client-Side Authorization, the session restriction was ultimately enforced through client-controlled runtime state.

.

4. Recommendations & Mitigations

The following recommendations are intended for developers seeking to strengthen native mobile applications against comparable reverse-engineering and binary-tampering techniques.

4.1 String Encryption & Control-Flow Obfuscation

Critical baseline parameters such as: `offline_session_duration` should not remain trivially discoverable as plaintext within `.rodata`. Compile-time or runtime string protection can increase the cost of static analysis. Control-flow obfuscation can also make straightforward XREF-based tracing substantially more difficult. Potential defensive approaches include:

- Sensitive-string encryption
- Runtime string reconstruction
- Symbol stripping
- Control-flow obfuscation
- Selective control-flow flattening
- Removal of unnecessary diagnostic strings

4.2 Dynamic Variable Scrambling

Sensitive runtime values—including wallet balances, entitlement states, and session-duration parameters—should not necessarily be maintained as straightforward plaintext integers in memory.

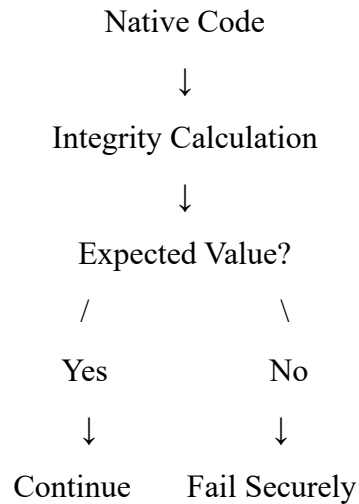
A defensive architecture can employ dynamically encoded representations such as: $\text{EncodedValue} = \text{Value} \text{ XOR } \text{RuntimeKey}$

with corresponding runtime decoding: $\text{Value} = \text{EncodedValue} \text{ XOR } \text{RuntimeKey}$

The key should ideally be generated or derived dynamically rather than embedded as another static constant.

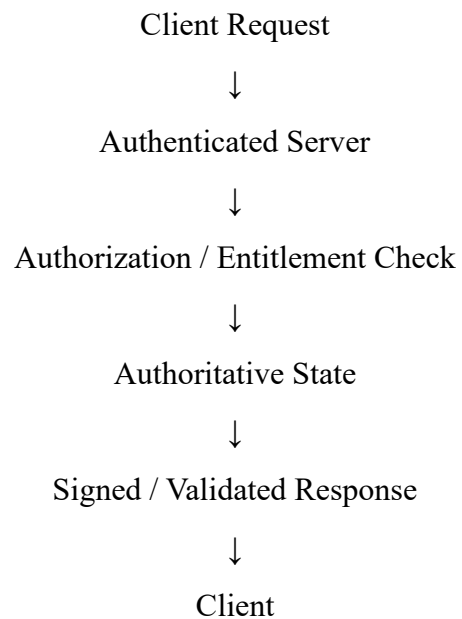
4.3 Runtime Anti-Tampering Checksums

Native code sections containing security-sensitive routines should be subject to integrity verification. A runtime integrity mechanism could periodically verify selected executable regions and compare the result against an expected integrity value. Conceptually:



4.4 Move Security-Critical Decisions Server-Side

The most important architectural mitigation is to avoid treating the client as the final authority for security-sensitive state. Premium currency, subscription entitlements, and account-level authorization should preferably follow a model such as:



5. Conclusion

This case study demonstrates how systematic ARM64 reverse engineering can expose security assumptions embedded within a mobile application's native execution environment. The most significant finding was that several security-relevant decisions were ultimately represented through client-controlled native code and runtime variables.

The research further demonstrated how ARM64 instruction semantics can materially affect application behavior. In particular, modification of an arithmetic operation such as: `NEG X20, X20` can change the sign of a transaction value, while an immediate assignment such as: `MOV W0, #0xFFFF` can substitute a configuration-derived runtime parameter.

From a security engineering perspective, these findings reinforce a central principle of mobile application security, obfuscation can increase the cost of reverse engineering, but it cannot make client-side state inherently trustworthy.

Overall Assessment, the analyzed architecture demonstrates a meaningful client-side trust boundary weakness, particularly where offline-session enforcement and sensitive entitlement state depend on locally modifiable native logic. Strengthening the client is useful for raising attack cost, but authoritative security decisions should ultimately be moved beyond the attacker's control wherever the product model permits.